

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

RAHUL MALLIKARJUN MATTI (1BM25CS498)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by **RAHUL MALLIKARJUN MATTI (1BM25CS498)** , who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026

. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF (Preemptive and Non-Preemptive) c) Priority (Preemptive and Non-Preemptive) d) Round Robin (Experiment with different quantum sizes for RR algorithm)	5-23
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	24-26
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	27-36
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	37-42
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	43-51
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	52-59

7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	60-69
----	---	-------

Course Outcomes:

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

LAB PROGRAM 1A)FCFS

```
#include <stdio.h>
int main() {
    printf("NAME:RAHUL M MATTI\n" "USN:1BM25CS498\n");
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int AT[n], BT[n], CT[n], TAT[n], WT[n], RT[n];
    int current_time = 0;
    float avgCT = 0, avgTAT = 0, avgWT = 0, avgRT = 0;
    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
        scanf("%d %d", &AT[i], &BT[i]);
    }
    for(i = 0; i < n; i++) {
        if(current_time < AT[i])
            current_time = AT[i];
        RT[i] = current_time - AT[i];
        CT[i] = current_time + BT[i];
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];
        current_time = CT[i];
        avgCT += CT[i];
        avgTAT += TAT[i];
        avgWT += WT[i];
        avgRT += RT[i];
    }
    printf("\n%-8s %-14s %-12s %-18s %-18s %-14s %-14s\n",
        "PROCESS", "ARRIVAL TIME", "BURST TIME", "COMPLETION TIME",
        "TURN AROUND TIME", "WAITING TIME", "RESPONSE TIME");
    for(i = 0; i < n; i++) {
        printf("P%-7d %-14d %-12d %-18d %-18d %-14d %-14d\n",
            i+1, AT[i], BT[i], CT[i], TAT[i], WT[i], RT[i]);
    }
    printf("\nAverage CT = %.2f", avgCT/n);
    printf("\nAverage TAT = %.2f", avgTAT/n);
    printf("\nAverage WT = %.2f", avgWT/n);
}
```

```

printf("\nAverage RT = %.2f\n", avgRT/n);
return 0;
}

```

OUTPUT:

FCFS(FIRST COME FIRST SERVE)

NAME:RAHUL M MATTI

USN:1BM25CS498

Enter number of processes: 3

Enter Arrival Time and Burst Time for P1: 0

6

Enter Arrival Time and Burst Time for P2: 0

8

Enter Arrival Time and Burst Time for P3: 0

9

PROCESS	ARRIVAL TIME	BURST TIME	COMPLETION TIME	TURN AROUND TIME	WAITING TIME	RESPONSE TIME
P1	0	6	6	6	0	0
P2	0	8	14	14	6	6
P3	0	9	23	23	14	14

Average CT = 14.33

Average TAT = 14.33

Average WT = 6.67

Average RT = 6.67

LAB PROGRAM 1b) SJF (Preemptive and Non-Preemptive)

```
#include <stdio.h>

int main() {
    int n, i, pos;
    printf("Enter number of processes: ");
    scanf("%d",&n);

    int AT[n], BT[n], CT[n], TAT[n], WT[n], RT[n], done[n];
    int current_time = 0, completed = 0;

    float avgCT=0, avgTAT=0, avgWT=0, avgRT=0;

    for(i=0;i<n;i++){
        printf("Enter AT and BT for P%d: ",i+1);
        scanf("%d %d",&AT[i],&BT[i]);
        done[i]=0;
    }

    while(completed < n){
        int minBT = 999;

        for(i=0;i<n;i++){
            if(AT[i] <= current_time && done[i]==0 && BT[i] < minBT){
                minBT = BT[i];
                pos = i;
            }
        }
    }
}
```

```

    }
    if(minBT == 999){
        current_time++;
    }
    else{
        RT[pos] = current_time - AT[pos];

        CT[pos] = current_time + BT[pos];
        TAT[pos] = CT[pos] - AT[pos];
        WT[pos] = TAT[pos] - BT[pos];

        current_time = CT[pos];
        done[pos] = 1;
        completed++;
        avgCT += CT[pos];
        avgTAT += TAT[pos];
        avgWT += WT[pos];
        avgRT += RT[pos];
    }
}

printf("\n%-5s %-12s %-10s %-15s %-15s %-12s %-12s\n",

"P", "ArrivalTime", "BurstTime", "CompletionTime", "TurnAroundTime", "WaitingTime", "
ResponseTime");

for(i=0; i<n; i++)

    printf("%-5s %-12d %-10d %-15d %-15d %-12d %-12d\n",

        "P", AT[i], BT[i], CT[i], TAT[i], WT[i], RT[i]);

```



```

printf("\nAverage CT = %.2f",avgCT/n);
printf("\nAverage TAT = %.2f",avgTAT/n);
printf("\nAverage WT = %.2f",avgWT/n);
printf("\nAverage RT = %.2f\n",avgRT/n);
return 0;
}

```

OUTPUT:

SJF(SHORTEST JOB FIRST)

NAME:RAHUL M MATTI

USN:1BM25CS498

Enter number of processes: 3

Enter AT and BT for P1: 0

4

Enter AT and BT for P2: 0

3

Enter AT and BT for P3: 0

8

P	ArrivalTime	BurstTime	CompletionTime	TurnAroundTime	WaitingTime	ResponseTime
P	0	4	7	7	3	3
P	0	3	3	3	0	0
P	0	8	15	15	7	7

Average CT = 8.33

Average TAT = 8.33

Average WT = 3.33

Average RT = 3.33

LAB PROGRAM 1B) SRTF(PREEMPTIVE)

```
#include <stdio.h>

int main() {

    printf("NAME:RAHUL M MATTI\n" "USN:1BM25CS498\n");

    int n;

    printf("Enter number of processes: ");

    scanf("%d",&n);

    int AT[n], BT[n], RT[n], CT[n], TAT[n], WT[n], start[n];

    int i, time = 0, completed = 0, min, pos;

    float avgCT=0, avgTAT=0, avgWT=0, avgRT=0;

    for(i=0;i<n;i++){

        printf("Enter AT and BT for P%d: ",i+1);

        scanf("%d %d",&AT[i],&BT[i]);

        RT[i] = BT[i];

        start[i] = -1;

    }

    while(completed < n){

        min = 999;

        for(i=0;i<n;i++){

            if(AT[i] <= time && RT[i] > 0 && RT[i] < min){

                min = RT[i];

                pos = i;

            }

        }

        if(min == 999){

            time++;

        }

    }

}
```

```

else{
    if(start[pos] == -1)
        start[pos] = time;
    RT[pos]--;
    time++;
    if(RT[pos] == 0){
        completed++;
        CT[pos] = time;
        TAT[pos] = CT[pos] - AT[pos];
        WT[pos] = TAT[pos] - BT[pos];
        avgCT += CT[pos];
        avgTAT += TAT[pos];
        avgWT += WT[pos];
        avgRT += (start[pos] - AT[pos]);
    }
}

printf("\n%-5s %-12s %-10s %-15s %-15s %-12s %-12s\n",
"P","ArrivalTime","BurstTime","CompletionTime","TurnAroundTime","WaitingTime","
ResponseTime");

for(i=0;i<n;i++)
    printf("%-5s %-12d %-10d %-15d %-15d %-12d %-12d\n",
"P", AT[i], BT[i], CT[i], TAT[i], WT[i], start[i]-AT[i]);

printf("\nAverage CT = %.2f",avgCT/n);
printf("\nAverage TAT = %.2f",avgTAT/n);
printf("\nAverage WT = %.2f",avgWT/n);
printf("\nAverage RT = %.2f\n",avgRT/n);

```

```
    return 0;
}
```

OUTPUT:

```
NAME:RAHUL M MATTI
USN:1BM25CS498
Enter number of processes: 3
Enter AT and BT for P1: 0
7
Enter AT and BT for P2: 0
6
Enter AT and BT for P3: 0
8

P    ArrivalTime  BurstTime  CompletionTime  TurnAroundTime  WaitingTime  ResponseTime
P    0             7           13              13              6            6
P    0             6           6               6              0            0
P    0             8          21             21             13           13

Average CT  = 13.33
Average TAT = 13.33
Average WT  = 6.33
Average RT  = 6.33
```

LAB PROGRAM 1C) PRIORITY SCHEDULING (PREEMPTIVE):

```
#include <stdio.h>

int main()
{
    int n,i;
    int pid[10],at[10],bt[10],pr[10],rt[10];
    int ct[10],tat[10],wt[10];
    int completed=0,current_time=0,min;
    float avg_tat=0,avg_wt=0;
    printf("\n\tNAME : RAHUL M MATTI\tUSN : 1BM25CS498\tSECTION : 'O'\n\n");
    printf("Enter number of processes: ");
    scanf("%d",&n);
    printf("Enter Arrival Time:\n");
    for(i=0;i<n;i++)
    {
        pid[i]=i+1;
        scanf("%d",&at[i]);
    }
    printf("Enter Burst Time:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&bt[i]);
        rt[i]=bt[i];
    }

    printf("Enter Priority:\n");
    for(i=0;i<n;i++)
```

```

scanf("%d",&pr[i]);
while(completed<n)
{
    min=-1;
    for(i=0;i<n;i++)
    {
        if(at[i]<=current_time && rt[i]>0)
        {
            if(min==-1 || pr[i] < pr[min])
                min=i;
        }
    }
    if(min==-1)
        current_time++;
    else
    {
        rt[min]--;
        current_time++;
        if(rt[min]==0)
        {
            ct[min]=current_time;
            tat[min]=ct[min]-at[min];
            wt[min]=tat[min]-bt[min];
            completed++;
            avg_tat+=tat[min];
            avg_wt+=wt[min];
        }
    }
}

```

```

    }
}
printf("\nPriority Scheduling (Preemptive) Higher the Number, Higher the Priority\n");
printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
for(i=0;i<n;i++)
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);
return 0;
}

```

OUTPUT:

PRIORITY SCHEDULING(PREEMPTIVE)

```
NAME : RAHUL M MATTI    USN : 1BM25CS498    SECTION : 'O'
Enter number of processes: 7
Enter Arrival Time:
0
1
3
4
5
6
10
Enter Burst Time:
8
2
4
1
6
5
1
Enter Priority:
3
4
4
5
2
6
1

Priority Scheduling (Preemptive) Higher the Number, Higher the Priority
PID    AT    BT    PR    CT    TAT    WT
P1      0      8      3     15     15      7
P2      1      2      4     17     16     14
P3      3      4      4     21     18     14
P4      4      1      5     22     18     17
P5      5      6      2     12      7      1
P6      6      5      6     27     21     16
P7     10      1      1     11      1      0

Average Turnaround Time = 13.71
Average Waiting Time = 9.86
```


LAB PROGRAM 1C)PRIORITY SCHEDULING (NON PREEMPTIVE)

```
#include <stdio.h>

int main()
{
    int n,i;
    int pid[10],at[10],bt[10],pr[10];
    int ct[10],tat[10],wt[10];
    int finished[10]={0};
    int current_time=0,completed=0,max;
    float avg_tat=0,avg_wt=0;
    printf("\n\tNAME : RAHUL M MATTI\tUSN : 1BM25CS498\tSECTION : 'O'\n\n");
    printf("Enter number of processes: ");
    scanf("%d",&n);
    printf("Enter Arrival Time:\n");
    for(i=0;i<n;i++)
    {
        pid[i]=i+1;
        scanf("%d",&at[i]);
    }
    printf("Enter Burst Time:\n");
    for(i=0;i<n;i++)
        scanf("%d",&bt[i]);
    printf("Enter Priority:\n");
    for(i=0;i<n;i++)
        scanf("%d",&pr[i]);
    while(completed<n)
    {
```

```

max=-1;
for(i=0;i<n;i++)
{
    if(at[i]<=current_time && finished[i]==0)
    {
        if(max==-1 || pr[i]>pr[max])
            max=i;
    }
}
if(max==-1)
    current_time++;
else
{
    ct[max]=current_time+bt[max];
    tat[max]=ct[max]-at[max];
    wt[max]=tat[max]-bt[max];
    current_time=ct[max];
    finished[max]=1;
    completed++;
    avg_tat+=tat[max];
    avg_wt+=wt[max];
}
}
printf("\nPriority Scheduling (Higher number = Higher priority)\n");
printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
for(i=0;i<n;i++)
{

```

```

        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
    printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);
    return 0;
}

```

OUTPUT:

PRIORITY SCHEDULING(NON PREEMPTIVE)

```

NAME : RAHUL M MATTI      USN : 1BM25CS498      SECTION : 'O'
Enter number of processes: 5
Enter Arrival Time:
0
2
3
4
6
Enter Burst Time:
3
2
5
4
1
Enter Priority:
5
3
2
4
1
Priority Scheduling (Preemptive) Higher the Number, Higher the Priority
PID    AT    BT    PR    CT    TAT    WT
P1      0      3      5     15     15     12
P2      2      2      3     10      8      6
P3      3      5      2      9      6      1
P4      4      4      4     14     10      6
P5      6      1      1      7      1      0
Average Turnaround Time = 8.00
Average Waiting Time = 5.00

```

LAB PROGRAM 1D) Round Robin (Experiment with different quantum sizes for RR algorithm)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i;
```

```
    int pid[10],at[10],bt[10],rt[10];
```

```
    int ct[10],tat[10],wt[10];
```

```
    int tq,current_time=0,completed=0;
```

```
    float avg_tat=0,avg_wt=0;
```

```
    printf("\n\tNAME : RAHUL M MATTI\tUSN : 1BM25CS498\tSECTION : 'O'\n\n");
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter Arrival Time:\n");
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        pid[i]=i+1;
```

```
        scanf("%d",&at[i]);
```

```
    }
```

```
    printf("Enter Burst Time:\n");
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        scanf("%d",&bt[i]);
```

```
        rt[i]=bt[i];
```

```
    }
```

```
    printf("Enter Time Quantum: ");
```

```
    scanf("%d",&tq);
```

```

int done;
while(completed<n)
{
    done=1;
    for(i=0;i<n;i++)
    {
        if(at[i]<=current_time && rt[i]>0)
        {
            done=0;
            if(rt[i] > tq)
            {
                current_time += tq;
                rt[i] -= tq;
            }
            else
            {
                current_time += rt[i];
                rt[i]=0;
                ct[i]=current_time;
                tat[i]=ct[i]-at[i];
                wt[i]=tat[i]-bt[i];
                completed++;
                avg_tat+=tat[i];
                avg_wt+=wt[i];
            }
        }
    }
}

```

```

        if(done==1)
        {
            current_time++;
        }
    }
    printf("\nRound Robin Scheduling\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
    printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);

    return 0;
}

```

OUTPUT:

ROUND ROBIN

```
NAME : RAHUL M MATTI    USN : 1BM25CS498    SECTION : 'O'

Enter number of processes: 5
Enter Arrival Time:
0
1
2
3
4
Enter Burst Time:
5
3
1
2
3
Enter Time Quantum: 2

Round Robin Scheduling
PID    AT    BT    CT    TAT    WT
P1      0     5    14    14     9
P2      1     3    12    11     8
P3      2     1     5     3     2
P4      3     2     7     4     2
P5      4     3    13     9     6

Average Turnaround Time = 8.20
Average Waiting Time = 5.40
```

LAB PROGRAM 2. MULTI LEVEL QUEUE:

```
#include <stdio.h>

struct Process {
    int pid, at, bt, ct, tat, wt, type;
};

int main() {
    printf("\n\tNAME : RAHUL M MATTI\tUSN : 1BM25CS498\tSECTION : 'O'\n\n");
    int n, i, time = 0;
    struct Process p[20];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        printf("PID: ");
        scanf("%d", &p[i].pid);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Type (1-System, 2-User): ");
        scanf("%d", &p[i].type);
    }
    for(i = 0; i < n; i++) {
        if(p[i].type == 1) {
            if(time < p[i].at)
                time = p[i].at;
```



```

        time += p[i].bt;
        p[i].ct = time;
    }
}

for(i = 0; i < n; i++) {
    if(p[i].type == 2) {
        if(time < p[i].at)
            time = p[i].at;

        time += p[i].bt;
        p[i].ct = time;
    }
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
float avgTAT = 0, avgWT = 0;
for(i = 0; i < n; i++) {
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    avgTAT += p[i].tat;
    avgWT += p[i].wt;
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt,
        p[i].ct, p[i].tat, p[i].wt);
}

printf("\nAverage Turnaround Time = %.2f", avgTAT / n);
printf("\nAverage Waiting Time = %.2f\n", avgWT / n);

```

```
    return 0;
}
```

OUTPUT:

```
NAME : RAHUL M MATTI    USN : 1BM25CS498    SECTION : '0'

Enter number of processes: 4

Process 1
PID: 1
Arrival Time: 0
Burst Time: 4
Type (1-System, 2-User): 1

Process 2
PID: 2
Arrival Time: 0
Burst Time: 5
Type (1-System, 2-User): 1

Process 3
PID: 3
Arrival Time: 0
Burst Time: 8
Type (1-System, 2-User): 2

Process 4
PID: 4
Arrival Time: 10
Burst Time: 5
Type (1-System, 2-User): 1

PID    AT    BT    CT    TAT    WT
1       0     4     4     4     0
2       0     5     9     9     4
3       0     8    23    23    15
4      10     5    15     5     0

Average Turnaround Time = 10.25
Average Waiting Time = 4.75
```

LAB PROGRAM 3: Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

```
#include <stdio.h>
#include <math.h>
int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
int main() {
    int n, i;
    printf("\n\tNAME : RAHUL M MATTI\tUSN : 1BM25CS498\tSECTION : 'O'\n\n");
    printf("Enter the number of processes:");
    scanf("%d", &n);
    int bt[n], p[n];
    printf("Enter the CPU burst times:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &bt[i]);
    }
}
```

```

printf("Enter the time periods:\n");
for (i = 0; i < n; i++) {
    scanf("%d", &p[i]);
}
int hyper = p[0];
for (i = 1; i < n; i++) {
    hyper = lcm(hyper, p[i]);
}
printf("LCM=%d\n\n", hyper);
printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\n", i + 1, bt[i], p[i]);
}
float U = 0;
for (i = 0; i < n; i++) {
    U += (float)bt[i] / p[i];
}
float bound = n * (pow(2, (float)1/n) - 1);
printf("\n%f <= %f => %s\n",
    U, bound, (U <= bound) ? "true" : "false");
printf("Scheduling occurs for %d ms\n", hyper);
int rem[n];
for (i = 0; i < n; i++)
    rem[i] = 0;

printf("\n");

```

```

int prev = -2;
for (int time = 0; time < hyper; time++) {
    for (i = 0; i < n; i++) {
        if (time % p[i] == 0) {
            rem[i] = bt[i];
        }
    }
    int selected = -1;
    for (i = 0; i < n; i++) {
        if (rem[i] > 0) {
            if (selected == -1 || p[i] < p[selected]) {
                selected = i;
            }
        }
    }
    if (selected != prev) {
        if (selected == -1)
            printf("%dms onwards: CPU is idle\n", time);
        else
            printf("%dms onwards: Process %d running\n", time, selected + 1);
    }
    if (selected != -1) {
        rem[selected]--;
    }
    prev = selected;
}
return 0;

```

}

OUTPUT:

RATE MONOTONIC SCHEDULING

```
NAME : RAHUL M MATTI    USN : 1BM25CS498    SECTION : 'O'

Enter the number of processes:2
Enter the CPU burst times:
20
35
Enter the time periods:
50
100
LCM=100

Rate Monotone Scheduling:
PID      Burst   Period
1         20      50
2         35      100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle
```

LAB PROGRAM 3b:EARLIEST DEADLINE FIRST

```
#include <stdio.h>

int main() {
    int n, i;

    printf("\n\tNAME : RAHUL M MATTI\tUSN : 1BM25CS498\tSECTION : 'O'\n\n");
    printf("Enter number of tasks: ");
    scanf("%d", &n);

    int bt[n], dl[n], p[n], rem[n];
    printf("Enter Burst times:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &bt[i]);
        rem[i] = 0;
    }

    printf("Enter Deadlines:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &dl[i]);
    }

    printf("Enter Periods:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &p[i]);
    }

    printf("\nPID\tBurst\tDeadline\tPeriod\n");
    for (i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\n", i + 1, bt[i], dl[i], p[i]);
    }

    int total_time = p[0];
    for (i = 1; i < n; i++) {
```

```

    if (p[i] > total_time)
        total_time = p[i];
}
printf("\nScheduling occurs for %d ms\n\n", total_time);
int prev_idle = 0;
for (int time = 0; time < total_time; time++) {
    for (i = 0; i < n; i++) {
        if (time % p[i] == 0) {
            rem[i] = bt[i];
        }
    }
    int selected = -1;
    int min_deadline = 9999;
    for (i = 0; i < n; i++) {
        if (rem[i] > 0) {
            if (dl[i] < min_deadline) {
                min_deadline = dl[i];
                selected = i;
            }
        }
    }
    if (selected == -1) {
        if (!prev_idle) {
            printf("%dms onwards : CPU is idle.\n", time);
            prev_idle = 1;
        }
    } else {

```



```

        printf("%dms : Task %d is running.\n", time, selected + 1);
        rem[selected]--;
        prev_idle = 0;
    }
}
return 0;
}

```

OUTPUT:

EARLIEST DEADLINE SCHEDULING

```

NAME : RAHUL M MATTI      USN : 1BM25CS498      SECTION : 'O'
Enter number of tasks: 3
Enter Burst times:
2
3
2
Enter Deadlines:
4
7
8
Enter Periods:
5
20
10

PID      Burst   Deadline   Period
1         2         4          5
2         3         7         20
3         2         8         10

Scheduling occurs for 20 ms

0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 1 is running.
6ms : Task 1 is running.
7ms : Task 3 is running.
8ms : Task 3 is running.
9ms onwards : CPU is idle.
10ms : Task 1 is running.
11ms : Task 1 is running.
12ms : Task 3 is running.
13ms : Task 3 is running.
14ms onwards : CPU is idle.
15ms : Task 1 is running.
16ms : Task 1 is running.
17ms onwards : CPU is idle.

```

LAB PROGRAM 3c:PROPORTIONAL SHARING

```
#include<stdio.h>
#include <stdlib.h>
#include <time.h>
typedef struct
{
char name[5]; int tickets; }
Process;
int main() {
int n, total_tickets = 0; float total_T =0.0;
printf("Enter the number of Processes: ");
scanf("%d", &n);
Process p[n];
srand(time(NULL));
for (int i = 0; i < n; i++)
{ printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: ");
scanf("%d", &p[i].tickets);
total_tickets += p[i].tickets;
total_T +=p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;
scanf("%d",&m);
for (int i = 0; i < m; i++)
```

```

{
int winning_ticket = rand() % total_tickets + 1;
int accumulated_tickets = 0;
int winner_index;
for (int j = 0; j < n; j++)
{
accumulated_tickets += p[j].tickets;
if (winning_ticket <= accumulated_tickets)
{
winner_index = j; break;
}
}
printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
p[winner_index].name);
}
for (int i = 0; i < n; i++)
{
printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
((p[i].tickets / total_T) * 100));
} return 0;
}

```

OUTPUT:

PROPORTIONAL SHARE SCHEDULING

```
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 29, Winner: P2
Tickets picked: 12, Winner: P2
Tickets picked: 59, Winner: P3
Tickets picked: 53, Winner: P3
Tickets picked: 53, Winner: P3

The Process: P1 gets 16.67% of Processor Time.
The Process: P2 gets 33.33% of Processor Time.
The Process: P3 gets 50.00% of Processor Time.
```

LAB PROGRAM 4a: PRODUCER CONSUMER PROBLEM USING SEMAPHORES

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty, full, mutex;
void producer() {
    int item;
    if (sem_trywait(&empty) != 0) {
        printf("Buffer is FULL! Cannot produce.\n");
        return;
    }
    sem_wait(&mutex);
    printf("Enter item to produce: ");
    scanf("%d", &item);
    buffer[in] = item;
    printf("Produced %d at position %d\n", item, in);
    in = (in + 1) % BUFFER_SIZE;
    sem_post(&mutex);
    sem_post(&full);
}
void consumer() {
    int item;
    if (sem_trywait(&full) != 0) {
        printf("Buffer is EMPTY! Cannot consume.\n");
        return;
    }
    sem_wait(&mutex);
    item = buffer[out];
    printf("Consumed %d from position %d\n", item, out);
    out = (out + 1) % BUFFER_SIZE;
    sem_post(&mutex);
    sem_post(&empty);
}
int main() {
    int choice;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    printf("Enter ");
    printf("1. Producer ");
    printf("2. Consumer ");
```

```
printf("3. Exit\n");
while (1) {
    printf("\nEnter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            producer();
            break;
        case 2:
            consumer();
            break;
        case 3:
            printf("Exiting...\n");
            sem_destroy(&empty);
            sem_destroy(&full);
            sem_destroy(&mutex);
            exit(0);
        default:
            printf("Invalid choice!\n");
    }
}
return 0;
}
```

```
Enter 1. Producer 2. Consumer 3. Exit
```

```
Enter choice: 1
```

```
Enter item to produce: 1
```

```
Produced 1 at position 0
```

```
Enter choice: 1
```

```
Enter item to produce: 2
```

```
Produced 2 at position 1
```

```
Enter choice: 1
```

```
Enter item to produce: 3
```

```
Produced 3 at position 2
```

```
Enter choice: 2
```

```
Consumed 1 from position 0
```

```
Enter choice: 2
```

```
Consumed 2 from position 1
```

```
Enter choice: 2
```

```
Consumed 3 from position 2
```

```
Enter choice: 2
```

```
Buffer is EMPTY! Cannot consume.
```

```
Enter choice: 3
```

```
Exiting...
```

LAB PROGRAM 4b:DINING PHILOSOPHER'S PROBLEM

```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num){
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;
    while (1){
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        if (id % 2 == 0){
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        }else{
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
            pthread_mutex_lock(&forks[left]); // FIXED LINE
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }
        printf("Philosopher %d is eating.\n", id);
```



```

        sleep(2);
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }
    return NULL;
}

int main(){
    int i;
    int ids[N];
    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
    for (i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }
    for (i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL);
    }
    for (i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }
}

```

```
}  
    return 0;  
}
```

OUTPUT:

```
Philosopher 2 is thinking.  
Philosopher 0 is thinking.  
Philosopher 3 is thinking.  
Philosopher 1 is thinking.  
Philosopher 4 is thinking.  
Philosopher 1 picked up right fork 2  
Philosopher 1 picked up left fork 1.  
Philosopher 1 is eating.  
Philosopher 3 picked up right fork 4  
Philosopher 3 picked up left fork 3.  
Philosopher 0 picked up left fork 0.  
Philosopher 3 is eating.  
Philosopher 1 put down forks 1 and 2  
Philosopher 1 is thinking.  
Philosopher 2 picked up left fork 2.  
Philosopher 0 picked up right fork 1  
Philosopher 0 is eating.  
Philosopher 3 put down forks 3 and 4  
Philosopher 3 is thinking.  
Philosopher 4 picked up left fork 4.  
Philosopher 2 picked up right fork 3  
Philosopher 2 is eating.  
|
```

LAB PROGRAM 5a: Bankers' algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
```

```
int main() {  
    int n, m, i, j, k;  
    printf("---Bankers algorithm--- \n");  
    printf("Enter number of processes: ");  
    scanf("%d", &n);  
    printf("Enter number of resources: ");  
    scanf("%d", &m);  
    int alloc[n][m], max[n][m], need[n][m];  
    int avail[m];  
    printf("Enter Allocation Matrix:\n");  
    for(i = 0; i < n; i++) {  
        for(j = 0; j < m; j++) {  
            scanf("%d", &alloc[i][j]);  
        }  
    }  
    printf("Enter Maximum Demand Matrix:\n");  
    for(i = 0; i < n; i++) {  
        for(j = 0; j < m; j++) {  
            scanf("%d", &max[i][j]);  
        }  
    }  
    printf("Enter Available Resources:\n");  
    for(i = 0; i < m; i++) {  
        scanf("%d", &avail[i]);  
    }  
}
```

```

}
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}
int finish[n], safeSeq[n], work[m];
for(i = 0; i < n; i++)
    finish[i] = 0;
for(i = 0; i < m; i++)
    work[i] = avail[i];
int count = 0;
while(count < n) {
    int found = 0;
    for(i = 0; i < n; i++) {
        if(finish[i] == 0) {
            int possible = 1;
            for(j = 0; j < m; j++) {
                if(need[i][j] > work[j]) {
                    possible = 0;
                    break;
                }
            }
            if(possible) {
                for(k = 0; k < m; k++)
                    work[k] += alloc[i][k];
            }
        }
    }
    count++;
}

```

```

        safeSeq[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}
}
if(found == 0)
    break;
}
if(count == n) {
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");
    for(i = 0; i < n; i++) {
        printf("P%d", safeSeq[i]);
        if(i != n - 1)
            printf(" -> ");
    }
    printf("\n");
}
else {
    printf("\nSystem is NOT in a safe state.\n");
}
return 0;
}

```

OUTPUT:

BANKERS ALGORITHM:

---Bankers algorithm---

Enter number of processes:

5

Enter number of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Demand Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resources:

3 3 2

System is in a safe state.

Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2

LAB PROGRAM 5b:DEADLOCK DETECTION

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;
    printf("\n---Deadlock Detection algorithm--- \n");
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the number of resources: ");
    scanf("%d", &m);
    int allocation[n][m];
    int request[n][m];
    int available[m];
    printf("\nEnter the allocation matrix:\n");
    for(i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }
    printf("\nEnter the request matrix:\n");
    for(i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++) {
            scanf("%d", &request[i][j]);
        }
    }
}
```

```

printf("\nEnter the available resources: ");
for(i = 0; i < m; i++) {
    scanf("%d", &available[i]);
}
int work[m];
int finish[n];
int safeSeq[n];
for(i = 0; i < m; i++)
    work[i] = available[i];
for(i = 0; i < n; i++) {
    int zero = 1;
    for(j = 0; j < m; j++) {
        if(allocation[i][j] != 0) {
            zero = 0;
            break;
        }
    }
    if(zero)
        finish[i] = 1;
    else
        finish[i] = 0;
}
int count = 0;
while(count < n) {
    int found = 0;
    for(i = 0; i < n; i++) {

```



```

    if(finish[i] == 0) {
        int possible = 1;
        for(j = 0; j < m; j++) {
            if(request[i][j] > work[j]) {
                possible = 0;
                break;
            }
        }
        if(possible) {
            for(k = 0; k < m; k++)
                work[k] += allocation[i][k];
            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }
}

if(found == 0)
    break;
}

int deadlock = 0;
for(i = 0; i < n; i++) {
    if(finish[i] == 0) {
        deadlock = 1;
        break;
    }
}

```

```

    }
    if(deadlock == 0) {
        printf("\nSystem is in safe state.\n");
        printf("Safe Sequence is: ");
        for(i = 0; i < count; i++) {
            printf("P%d ", safeSeq[i]);
        }
        printf("\n");
    }
    else {
        printf("\nSystem is in deadlock state.\n");
        printf("Deadlocked processes are: ");
        for(i = 0; i < n; i++) {
            if(finish[i] == 0)
                printf("P%d ", i);
        }
        printf("\n");
    }
    return 0;
}

```

OUTPUT:

DEADLOCK DETECTION:

---Deadlock Detection algorithm---

Enter the number of processes: 5

Enter the number of resources: 3

Enter the allocation matrix:

Process 0: 0 1 0

Process 1: 2 0 0

Process 2: 3 0 3

Process 3: 2 1 1

Process 4: 0 0 2

Enter the request matrix:

Process 0: 0 0 0

Process 1: 2 0 2

Process 2: 0 0 0

Process 3: 1 0 0

Process 4: 0 0 2

Enter the available resources: 0 0 0

System is in safe state.

Safe Sequence is: P0 P2 P3 P4 P1

LAB PROGRAM 6: Write a C program to simulate the following contiguous memory allocation techniques.

- a) Worst-fit
- b) Best-fit
- c) First-fit

CODE:

```
#include <stdio.h>

#define MAX 20

/* Function for First-Fit allocation */
void firstFit(int blocks[], int m, int process[], int n)
{
    int allocation[MAX];
    int i, j;

    for (i = 0; i < n; i++)
        allocation[i] = -1;

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++)
        {
            if (blocks[j] >= process[i])
            {
                allocation[i] = j;
                blocks[j] -= process[i];
                break;
            }
        }
    }
}
```

```

    }
}

printf("\n--- First Fit Allocation ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for (i = 0; i < n; i++)
{
    printf("%d\t%d\t", i + 1, process[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

/* Function for Best-Fit allocation */
void bestFit(int blocks[], int m, int process[], int n)
{
    int allocation[MAX];
    int i, j, bestIdx;

    for (i = 0; i < n; i++)
        allocation[i] = -1;

```

```

for (i = 0; i < n; i++)
{
    bestIdx = -1;

    for (j = 0; j < m; j++)
    {
        if (blocks[j] >= process[i])
        {
            if (bestIdx == -1 || blocks[j] < blocks[bestIdx])
                bestIdx = j;
        }
    }

    if (bestIdx != -1)
    {
        allocation[i] = bestIdx;
        blocks[bestIdx] -= process[i];
    }
}

printf("\n--- Best Fit Allocation ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for (i = 0; i < n; i++)
{
    printf("%d\t%d\t", i + 1, process[i]);

```

```

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

/* Function for Worst-Fit allocation */
void worstFit(int blocks[], int m, int process[], int n)
{
    int allocation[MAX];
    int i, j, worstIdx;

    for (i = 0; i < n; i++)
        allocation[i] = -1;

    for (i = 0; i < n; i++)
    {
        worstIdx = -1;

        for (j = 0; j < m; j++)
        {
            if (blocks[j] >= process[i])
            {
                if (worstIdx == -1 || blocks[j] > blocks[worstIdx])

```

```

        worstIdx = j;
    }
}

if (worstIdx != -1)
{
    allocation[i] = worstIdx;
    blocks[worstIdx] -= process[i];
}
}

printf("\n--- Worst Fit Allocation ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for (i = 0; i < n; i++)
{
    printf("%d\t%d\t", i + 1, process[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main()

```



```

{
    int m, n, i, choice;
    int blocks[MAX], process[MAX];
    int tempBlocks[MAX];

    printf("CONTIGUOUS MEMORY ALLOCATION TECHNIQUES\n");
    printf("1. First Fit\n");
    printf("2. Best Fit\n");
    printf("3. Worst Fit\n");

    printf("\nEnter number of memory blocks: ");
    scanf("%d", &m);

    printf("Enter sizes of memory blocks:\n");
    for (i = 0; i < m; i++)
        scanf("%d", &blocks[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter sizes of processes:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &process[i]);

    printf("\nEnter your choice: ");
    scanf("%d", &choice);

```

```
for (i = 0; i < m; i++)
    tempBlocks[i] = blocks[i];

switch (choice)
{
    case 1:
        firstFit(tempBlocks, m, process, n);
        break;

    case 2:
        bestFit(tempBlocks, m, process, n);
        break;

    case 3:
        worstFit(tempBlocks, m, process, n);
        break;

    default:
        printf("Invalid choice!\n");
}

return 0;
}
```

OUTPUT:

```
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426
```

--- First Fit Allocation ---

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	2
4	426	Not Allocated

--- Best Fit Allocation ---

Process No.	Process Size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

--- Worst Fit Allocation ---

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	5
4	426	Not Allocated

LAB PROGRAM 7a: Write a C program to simulate page replacement algorithms.

- a) FIFO
- b) LRU
- c) OPTIMAL

CODE:

```
#include <stdio.h>
```

```
#define MAX 50
```

```
/* Function to check whether page is already present */
```

```
int isPresent(int frames[], int f, int page)
```

```
{
```

```
    int i;
```

```
    for (i = 0; i < f; i++)
```

```
    {
```

```
        if (frames[i] == page)
```

```
            return 1;
```

```
    }
```

```
    return 0;
```

```
}
```

```
void fifo(int pages[], int n, int f)
```

```
{
```

```
    int frames[MAX], i, j, index = 0, faults = 0;
```

```
    for (i = 0; i < f; i++)
```

```
        frames[i] = -1;
```

```

printf("\n--- FIFO Page Replacement ---\n");
printf("Page\tFrames\n");
for (i = 0; i < n; i++)
{
    if (!isPresent(frames, f, pages[i]))
    {
        frames[index] = pages[i];
        index = (index + 1) % f;
        faults++;
    }
    printf("%d\t", pages[i]);
    for (j = 0; j < f; j++)
    {
        if (frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
    printf("\n");
}
printf("Total Page Faults = %d\n", faults);
}

```

```

int findLRU(int time[], int f)
{
    int i, pos = 0, min = time[0];

```

```

    for (i = 1; i < f; i++)
    {
        if (time[i] < min)
        {
            min = time[i];
            pos = i;
        }
    }
    return pos;
}

void lru(int pages[], int n, int f)
{
    int frames[MAX], time[MAX];
    int i, j, pos, faults = 0, counter = 0, found;
    for (i = 0; i < f; i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }
    printf("\n--- LRU Page Replacement ---\n");
    printf("Page\tFrames\n");
    for (i = 0; i < n; i++)
    {
        found = 0;
        for (j = 0; j < f; j++)

```

```

{
    if (frames[j] == pages[i])
    {
        counter++;
        time[j] = counter;
        found = 1;
        break;
    }
}
if (!found)
{
    pos = -1;
    for (j = 0; j < f; j++)
    {
        if (frames[j] == -1)
        {
            pos = j;
            break;
        }
    }
    if (pos == -1)
        pos = findLRU(time, f);
    frames[pos] = pages[i];
    counter++;
    time[pos] = counter;
    faults++;
}

```

```

    }
    printf("%d\t", pages[i]);
    for (j = 0; j < f; j++)
    {
        if (frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
    printf("\n");
}
printf("Total Page Faults = %d\n", faults);
}

int findOptimal(int pages[], int frames[], int n, int index, int f)
{
    int i, j, farthest = index, pos = -1;
    for (i = 0; i < f; i++)
    {
        int found = 0;
        for (j = index; j < n; j++)
        {
            if (frames[i] == pages[j])
            {
                if (j > farthest)
                {

```



```

        farthest = j;
        pos = i;
    }
    found = 1;
    break;
}
}
if (!found)
    return i;
}
if (pos == -1)
    return 0;
return pos;
}
void optimal(int pages[], int n, int f)
{
    int frames[MAX], i, j, pos, faults = 0;
    for (i = 0; i < f; i++)
        frames[i] = -1;
    printf("\n--- Optimal Page Replacement ---\n");
    printf("Page\tFrames\n");
    for (i = 0; i < n; i++)
    {
        if (!isPresent(frames, f, pages[i]))
        {
            pos = -1;

```

```

    for (j = 0; j < f; j++)
    {
        if (frames[j] == -1)
        {
            pos = j;
            break;
        }
    }
    if (pos == -1)
        pos = findOptimal(pages, frames, n, i + 1, f);
    frames[pos] = pages[i];
    faults++;
}
printf("%d\t", pages[i]);
for (j = 0; j < f; j++)
{
    if (frames[j] != -1)
        printf("%d ", frames[j]);
    else
        printf("- ");
}
printf("\n");
}
printf("Total Page Faults = %d\n", faults);
}

```

```

int main()
{
    int n, f, i, choice;
    int pages[MAX];
    printf("PAGE REPLACEMENT ALGORITHMS\n");
    printf("1. FIFO\n");
    printf("2. LRU\n");
    printf("3. Optimal\n");
    printf("\nEnter number of pages in reference string: ");
    scanf("%d", &n);
    printf("Enter the reference string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("\nEnter your choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
        case 1:
            fifo(pages, n, f);
            break;
        case 2:
            lru(pages, n, f);
            break;
        case 3:

```

```

        optimal(pages, n, f);
        break;
    default:
        printf("Invalid choice!\n");
    }
    return 0;
}

```

OUTPUT:

```

Enter number of pages in reference string: 13
Enter the reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2
Enter number of frames: 3

```

--- FIFO Page Replacement ---

Page	Frames
7	7 - -
0	7 0 -
1	7 0 1
2	2 0 1
0	2 0 1
3	2 3 1
0	2 3 0
4	4 3 0
2	4 2 0
3	4 2 3
0	0 2 3
3	0 2 3
2	0 2 3

Total Page Faults = 10

--- LRU Page Replacement ---

Page	Frames
7	7 - -
0	7 0 -
1	7 0 1
2	2 0 1
0	2 0 1
3	2 0 3
0	2 0 3
4	4 0 3
2	4 0 2
3	4 3 2
0	0 3 2
3	0 3 2
2	0 3 2

Total Page Faults = 9

--- Optimal Page Replacement ---

Page	Frames
------	--------

7	7 - -
---	-------

0	7 0 -
---	-------

1	7 0 1
---	-------

2	2 0 1
---	-------

0	2 0 1
---	-------

3	2 0 3
---	-------

0	2 0 3
---	-------

4	2 4 3
---	-------

2	2 4 3
---	-------

3	2 4 3
---	-------

0	2 0 3
---	-------

3	2 0 3
---	-------

2	2 0 3
---	-------

Total Page Faults = 7